

■ Self-Hosted VPN Server

Project Documentation

ocserv	Docker	Node.js	Nginx	SSL/TLS	Linux
--------	--------	---------	-------	---------	-------

By Ayman Ibousoufyane

June 2026

github.com/Ayman6594/VPN-Project

1. Introduction

This document covers the design, build, debugging, and deployment of a self-hosted SSL VPN server inspired by enterprise solutions like **FortiClient** (Fortinet) and **Ivanti Connect Secure**. Built entirely from scratch on a Windows machine using Docker, it includes a full web admin panel for user management and live monitoring.

What is a VPN?

A VPN creates an encrypted tunnel between a client device and a server. Enterprise SSL VPN products use TLS on port 443 — the same port as HTTPS — making connections nearly impossible to block by corporate firewalls or ISPs. This project uses **ocserv** (OpenConnect Server), which implements the same protocol as Cisco AnyConnect.

VPN Protocol Comparison

Protocol	Layer	Port	Used By
SSL VPN	Application	443/TCP	FortiClient, Ivanti, AnyConnect
IPsec VPN	Network	500/UDP	StrongSwan, Windows VPN
WireGuard	Network	51820/UDP	Modern self-hosted
OpenVPN	Application	1194/UDP	Community VPN projects

2. Architecture

Three Docker containers run on a shared internal network. A shared Docker volume (**vpn-data**) stores the TLS certificate, private key, and **ocpasswd** user database, making it accessible to both the VPN server and the admin panel.

Container Overview

Container	Base Image	Role	Port
vpn-server	Ubuntu 22.04	SSL VPN tunnel (ocserv)	443/TCP+UDP
vpn-web-ui	Node.js 20 Slim	Admin panel (Express)	3000/TCP
vpn-nginx	nginx:alpine	Reverse proxy	80, 8443

SSL VPN Connection Flow

1. Client connects to port **443 over TLS** (same as HTTPS). 2. Server sends its **TLS certificate** for authentication. 3. User provides **username + password** (stored in **ocpasswd**). 4. A virtual interface **vpns0** is created on both ends. 5. All traffic is encapsulated and encrypted in the tunnel. 6. Server does **NAT** so VPN clients can reach the internet.

Project File Structure

File / Folder	Purpose
docker-compose.yml	Orchestrates all 3 containers
.env	Admin credentials and session secret
ocserv/Dockerfile	Builds VPN container (Ubuntu + ocserv)
ocserv/ocserv.conf	VPN config: ports, IPs, DNS, auth method
ocserv/entrypoint.sh	Startup: generate certs, iptables, start ocserv
nginx/nginx.conf	Reverse proxy: HTTP→HTTPS, proxy to web-ui
web-ui/server.js	Express API: login, user CRUD, VPN status
web-ui/public/index.html	Single-page admin UI (vanilla JS + CSS)
web-ui/Dockerfile	Builds the Node.js admin container

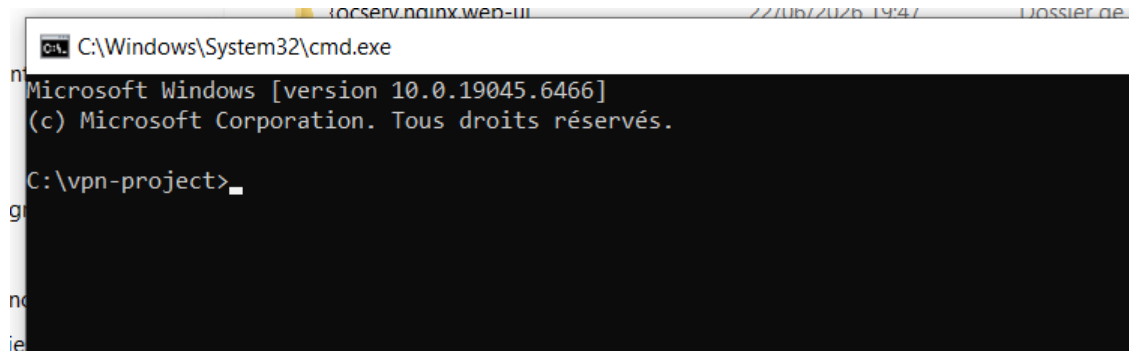
3. Step-by-Step Build Process

Below is every step taken to build, debug, and run the project on Windows with Docker Desktop.

1

Set up the project folder

Created the project at C:\vpn-project and opened a Command Prompt inside it.

A screenshot of a Windows Command Prompt window. The title bar shows 'C:\Windows\System32\cmd.exe'. The window content displays the Windows version '10.0.19045.6466' and the copyright notice '(c) Microsoft Corporation. Tous droits réservés.' The command prompt shows the current directory as 'C:\vpn-project' with a cursor at the end of the line.

```
C:\Windows\System32\cmd.exe
Microsoft Windows [version 10.0.19045.6466]
(c) Microsoft Corporation. Tous droits réservés.

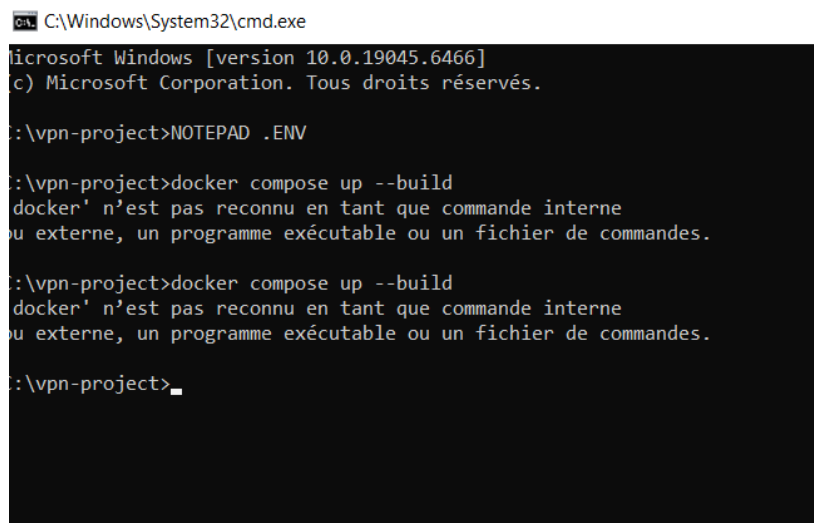
C:\vpn-project>
```

Fig 1 — Project folder in Windows Command Prompt

2

Configured environment variables

Edited .env using Notepad: set ADMIN_USER, ADMIN_PASS, and SESSION_SECRET.

A screenshot of a Windows Command Prompt window. The title bar shows 'C:\Windows\System32\cmd.exe'. The window content displays the Windows version '10.0.19045.6466' and the copyright notice '(c) Microsoft Corporation. Tous droits réservés.' The command prompt shows the current directory as 'C:\vpn-project'. The user has run 'NOTEPAD .ENV' and then 'docker compose up --build'. Both commands resulted in an error message: 'docker' n'est pas reconnu en tant que commande interne ou externe, un programme exécutable ou un fichier de commandes.

```
C:\Windows\System32\cmd.exe
Microsoft Windows [version 10.0.19045.6466]
(c) Microsoft Corporation. Tous droits réservés.

C:\vpn-project>NOTEPAD .ENV

C:\vpn-project>docker compose up --build
docker' n'est pas reconnu en tant que commande interne
ou externe, un programme exécutable ou un fichier de commandes.

C:\vpn-project>docker compose up --build
docker' n'est pas reconnu en tant que commande interne
ou externe, un programme exécutable ou un fichier de commandes.

C:\vpn-project>
```

Fig 2 — First docker compose up --build attempt

3

Fixed Alpine Linux package error

ocserv was not in Alpine 3.19 or 3.18 repos. Switched to Ubuntu 22.04 which has it.



Fixed web-ui package error

The web-ui Dockerfile also used Alpine. Switched to node:20-slim (Debian-based) to get ocpasswd tool.



Fig 5 — web-ui also failed on Alpine, fixed with node:20-slim

5

Fixed IP forwarding (read-only /proc in WSL2)

Docker Desktop on Windows uses WSL2 which blocks writes to `/proc/sys/net/ipv4/ip_forward`. Removed that line from `entrypoint.sh` and silenced iptables errors.

[illegible]

Fig 6 — vpn-server crash loop: /proc read-only filesystem

```

825 [web-ui] resolving provenance for metadata file
826 Done 0.0s
827 [cncf] exporting layers to Image
828 Exporting layers 0.2s done
829 exporting manifest sha256:0591f3ccfcb8c826b643d182808018e82146387764dc0c773af8e 0.0s done
830 exporting sha256:5a83a5b384540000000000000000000000000000000000000000000000000000 0.0s done
831 exporting attestation manifest sha256:19f0a701870707b24915d0e0a002159e61732391ac11b8514647d44 0.0s done
832 exporting sha256:0000000000000000000000000000000000000000000000000000000000000000 0.0s done
833 exporting manifest list sha256:5e7225a2608708080b643d182808018e82146387764dc0c773af8e 0.0s done
834 exporting sha256:0000000000000000000000000000000000000000000000000000000000000000 0.0s done
835 [cncf] impacting to docker://library/nginx-project-cncf:latest 0.1s done
836 Done 0.5s
837 [cncf] resolving provenance for metadata file
838 Done 0.0s
839 [cncf] /
840 [cncf] /
841 Image vsp-project-cncf/nginx:latest 2.9s
842 Image vsp-project-web-ui/nginx:latest 2.9s
843 Network vsp-project_vpn-net Created 0.2s
844 Container vsp-server Created 0.1s
845 Container vsp-web-ui Created 0.1s
846 Container vsp-engine Created 0.1s
847 Attaching to vsp-engine, vsp-web-ui, vsp-web-ui
848 vsp-engine => Starting VM Server Setup...
849 vsp-server => Starting services...
850 vsp-server note: whitelist: setting 'plain' as primary authentication method
851 vsp-server note: setting 'file' as supplemental config option
852 vsp-server listening (0.0 on 0.0.0.0.44)...
853 vsp-server listening (UDP on 0.0.0.0.44)...
854 vsp-engine [0]: sec mod: reading supplemental config from files
855 vsp-engine [0]: sec mod: sec mod initialized (config: /usr/local/etc-csnet-10ddeb.8)
856 vsp-engine /docker-entryptd.sh: /docker-entryptd.sh is not empty, I will attempt to perform configuration
857 vsp-engine /docker-entryptd.sh: Looking for shell scripts in /docker-entryptd.sh/
858 Done 0.0s
859 vsp-engine [0]: Launching /docker-entryptd.sh /D10-listen on /opt-by-default.sh
860 vsp-engine VMX Admin UI running on http://localhost:3000
861 View Docker Desktop View Config View Logs View Logs View Logs
862 vsp-engine 10-listen on /opt-by-default.sh: Info: getting the checksum of /etc/nginx/conf.d/default.conf
863 vsp-engine 10-listen on /opt-by-default.sh: Info: getting the checksum of /etc/nginx/conf.d/default.conf
864 vsp-engine [0]: Starting /docker-entryptd.sh /D15-local-responses_emoji
865 vsp-engine /docker-entryptd.sh: Launching /docker-entryptd.sh /D10-startup on templates.sh
866 vsp-engine /docker-entryptd.sh: Launching /docker-entryptd.sh /D10-time-worker-process.sh
867 vsp-engine /docker-entryptd.sh: Configuration complete; ready for start-up
868 Done 0.0s
869 View in Docker Desktop View Config View Logs Enable Watch Refresh

```

Fig 7 — All 6/6 containers built and running after the fix

6

Exposed the admin panel port

Port 443 was occupied by ocserv (the VPN itself). Added port 3000:3000 to web-ui in docker-compose.yml.

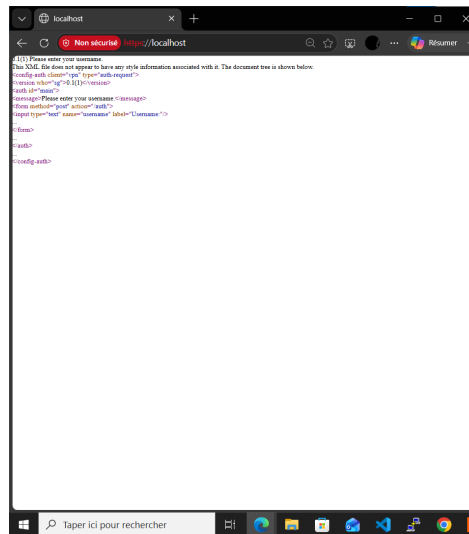


Fig 8 — localhost showed ocserv XML auth (port 443 taken by VPN)

4. The Admin Panel

A single-page admin application built with **Express.js** on the backend and **vanilla JavaScript + custom CSS** on the frontend. Accessible at <http://localhost:3000>.

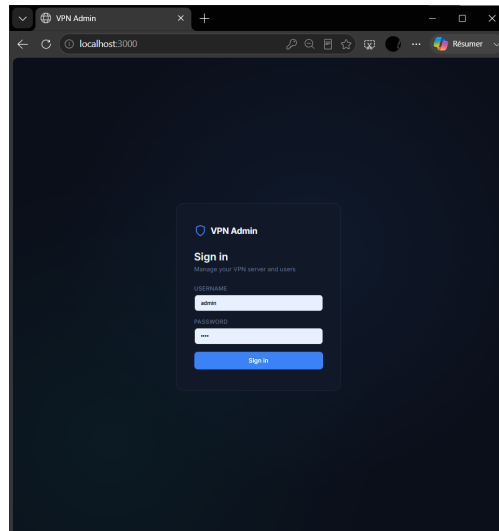


Fig 9 — VPN Admin login page at localhost:3000

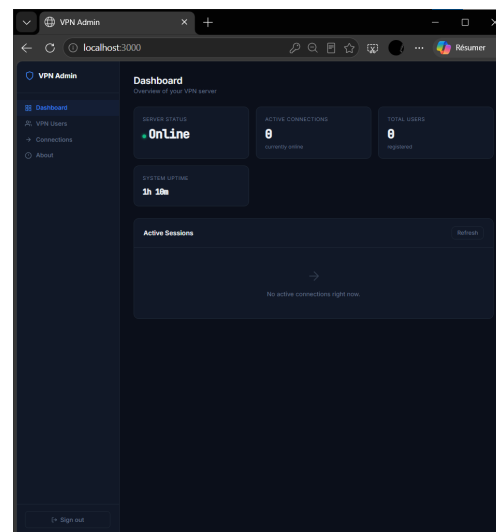


Fig 10 — Dashboard: Server Online, uptime 1h 19m

Admin Panel Features

Page	What it does
Dashboard	Server status, active connections, total users, system uptime
VPN Users	Create and delete VPN accounts stored in ocpasswd file
Connections	Live list of currently connected VPN clients
About	Project tech stack and architecture overview

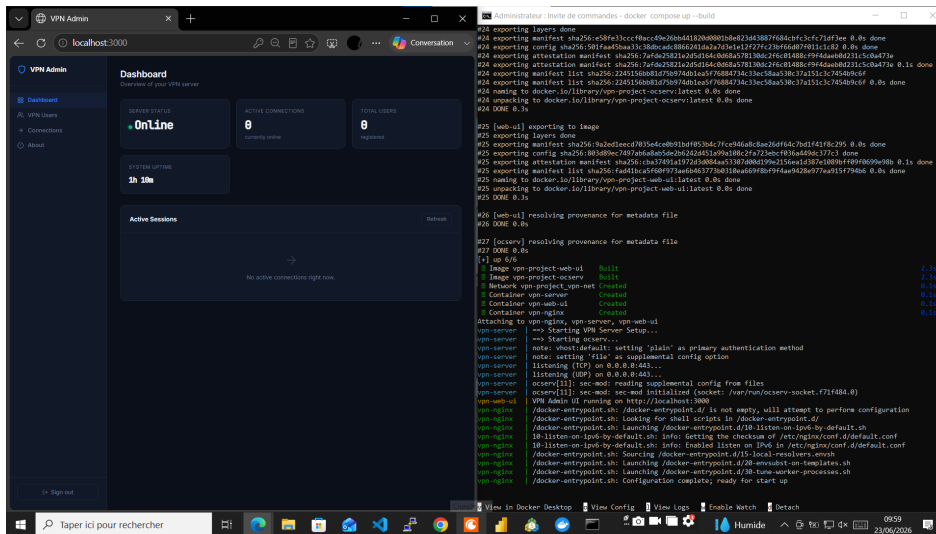


Fig 11 — Dashboard + running terminal side by side

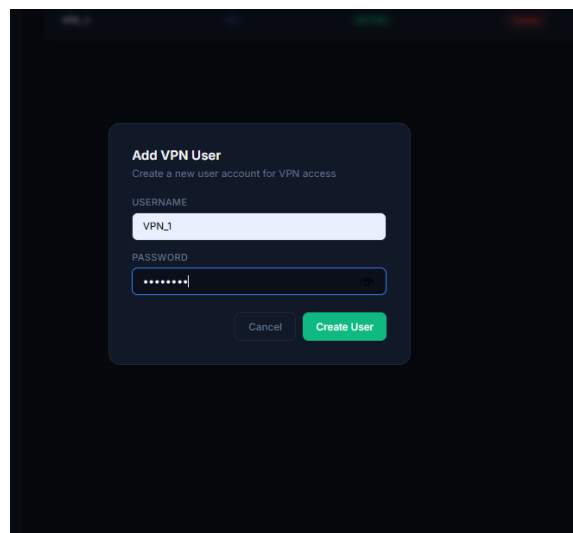


Fig 12 — Adding VPN user "VPN_1" via the admin panel

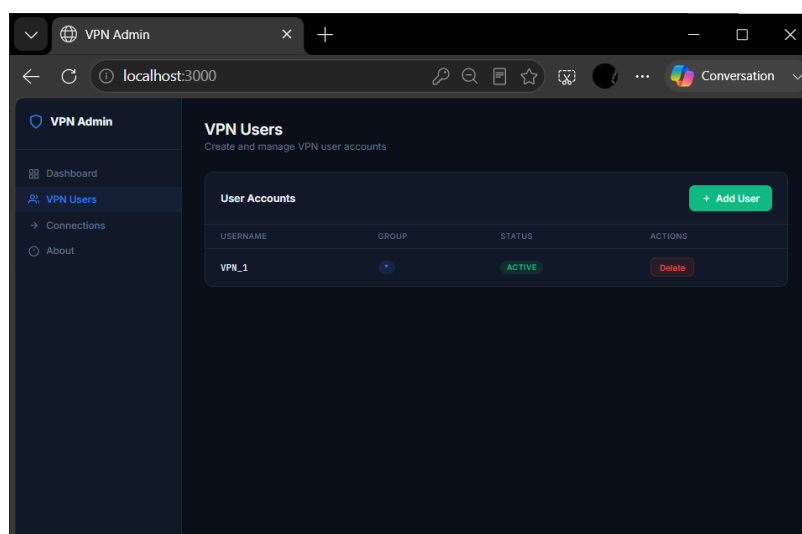
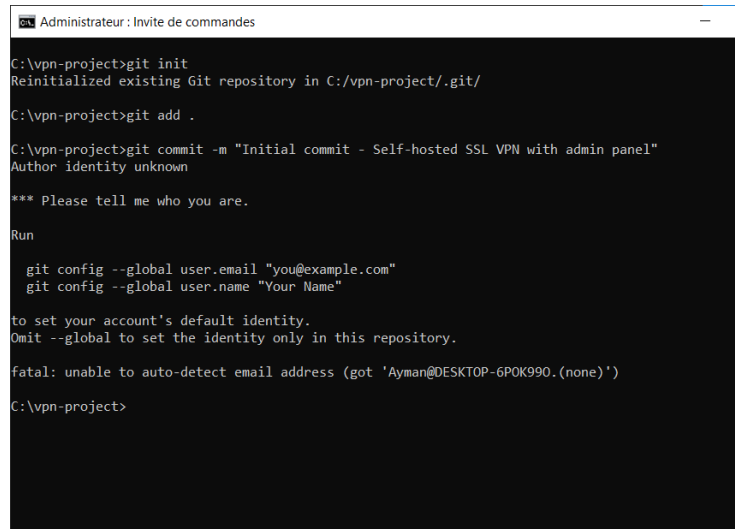


Fig 13 — VPN Users page showing VPN_1 as Active

5. Publishing to GitHub

The project was published as a public repository for portfolio and version control. Git was initialized, 11 files committed, and pushed using a Personal Access Token (GitHub no longer accepts account passwords for git push).



```

Administrateur : Invite de commandes

C:\vpn-project>git init
Reinitialized existing Git repository in C:/vpn-project/.git/

C:\vpn-project>git add .

C:\vpn-project>git commit -m "Initial commit - Self-hosted SSL VPN with admin panel"
Author identity unknown

*** Please tell me who you are.

Run

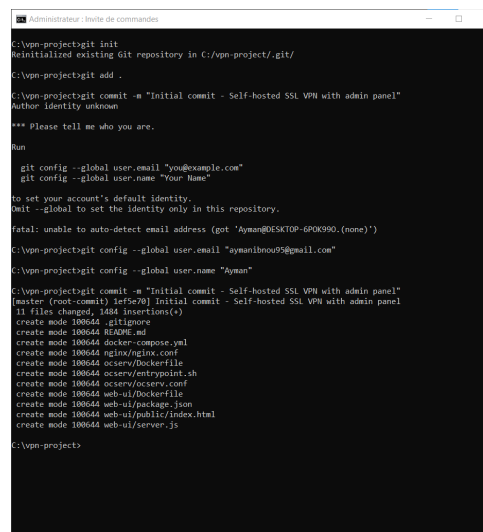
  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'Ayman@DESKTOP-6POK990.(none)')

C:\vpn-project>
  
```

Fig 14 — Git identity setup and initial commit (11 files)



```

Administrateur : Invite de commandes

C:\vpn-project>git init
Reinitialized existing Git repository in C:/vpn-project/.git/

C:\vpn-project>git add .

C:\vpn-project>git commit -m "Initial commit - Self-hosted SSL VPN with admin panel"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'Ayman@DESKTOP-6POK990.(none)')

C:\vpn-project>git config --global user.email "aymanibrou9@gmail.com"
C:\vpn-project>git config --global user.name "Ayman"

C:\vpn-project>git commit -m "Initial commit - Self-hosted SSL VPN with admin panel"
[master (root-commit) 1ef5e70] Initial commit - Self-hosted SSL VPN with admin panel
11 files changed, 1444 insertions(+), 0 deletions(-)
create mode 100644 .gitignore
create mode 100644 README.md
create mode 100644 docker-compose.yml
create mode 100644 nginx/nginx.conf
create mode 100644 ocserv/Dockerfile
create mode 100644 ocserv/entrypoint.sh
create mode 100644 ocserv/ocserv.conf
create mode 100644 web-ui/Dockerfile
create mode 100644 web-ui/package.json
create mode 100644 web-ui/public/index.html
create mode 100644 web-ui/server.js

C:\vpn-project>
  
```

Fig 15 — Successful commit after identity config

```
Administrateur: invite de commandes

to set your account's default identity.
git --global to set the identity only in this repository.

(fatal: unable to auto-detect email address (got 'Ayman@DESKTOP-6PK0990.(none)')

C:\vpn-project>git config --global user.email "aymanibrou59@gmail.com"

C:\vpn-project>git config --global user.name "Ayman"

C:\vpn-project>git commit -m "Initial commit - Self-hosted SSL VPN with admin panel"
[master (root-commit) 1ef5e70] Initial commit - Self-hosted SSL VPN with admin panel
11 files changed, 1484 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md
 create mode 100644 docker-compose.yml
 create mode 100644 nginx/nginx.conf
 create mode 100644 ocserv/Dockerfile
 create mode 100644 ocserv/entrypoint.sh
 create mode 100644 ocserv/ocserv.conf
 create mode 100644 web-ui/Dockerfile
 create mode 100644 web-ui/package.json
 create mode 100644 web-ui/public/index.html
 create mode 100644 web-ui/server.js

C:\vpn-project>git remote add origin https://github.com/ayman6594/vpn-project.git

C:\vpn-project>git branch -M main

C:\vpn-project>git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 4 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (17/17), 34.14 KiB | 2.36 MiB/s, done.
Total 17 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: This repository moved. Please use the new location:
remote: https://github.com/ayman6594/vpn-project.git
To https://github.com/ayman6594/vpn-project.git
 * [new branch] main -> main
branch 'main' set up to track 'origin/main'.

C:\vpn-project>
```

Fig 16 — git push: 17 objects pushed to GitHub

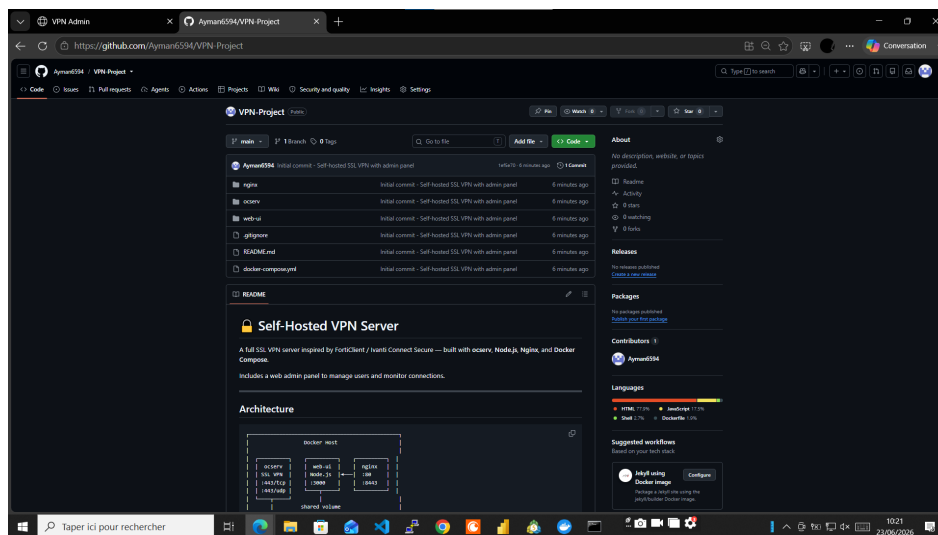


Fig 17 — Live GitHub repo: Ayman6594/VPN-Project

Repository: <https://github.com/Ayman6594/VPN-Project>

Languages detected: HTML 77.9% · JavaScript 17.5% · Shell 2.7% · Dockerfile 1.9%

6. Challenges Overcome

Package unavailability: ocserv was missing from Alpine Linux 3.18 and 3.19. Fixed by switching both containers to Debian/Ubuntu-based images.

Read-only /proc in WSL2: Docker Desktop on Windows uses WSL2, which blocks writes to /proc. Removed the ip_forward line and silenced iptables errors with || true.

Port 443 conflict: ocserv occupies port 443, so Nginx could not serve the admin panel there. Fixed by exposing port 3000 directly from the web-ui container.

Git authentication: GitHub dropped password auth for git push. Generated a Personal Access Token with repo scope and used it as the password.

7. What I Learned

Area	Concepts Covered
SSL/TLS VPN	TLS tunnels, self-signed certs with OpenSSL, AnyConnect protocol
Docker	Multi-container Compose, shared volumes, port mapping, networking
Linux Networking	iptables, NAT/MASQUERADE, IP forwarding, virtual interfaces
Node.js/Express	REST API, session auth, child_process for shell commands
Nginx	Reverse proxy, HTTP→HTTPS redirect, upstream config
Debugging	Reading Docker logs, distro differences, WSL2 kernel limits
Git / GitHub	Init, commit, remote setup, token-based push authentication

8. Future Improvements

1	Deploy to Oracle Cloud Free Tier Run on a real public server with a public IP, accessible from anywhere.
2	Let's Encrypt SSL Certificate Replace the self-signed cert with a trusted one using Certbot — no more browser warnings.
3	Two-Factor Authentication Add TOTP-based 2FA via PAM module for stronger user security.
4	Connection Logging Store VPN session history (timestamps, data usage) in SQLite or PostgreSQL.
5	Real-Time Traffic Dashboard Live bandwidth charts per user using Chart.js and WebSocket updates.
6	User Groups & Access Policies Different network access levels per group (e.g. admin vs standard user).

9. Conclusion

This project demonstrates the end-to-end implementation of an enterprise-grade SSL VPN server built entirely from scratch. Starting from understanding how FortiClient and Ivanti work under the hood, each layer was built and debugged: VPN core (ocserv), admin interface (Node.js), reverse proxy (Nginx), and full containerization (Docker Compose). Every error was a real learning opportunity.

Project Summary

Self-Hosted SSL VPN Server with Web Admin Panel
Author: **Ayman Ibrousoufyane** | June 2026
Stack: ocserv · Ubuntu 22.04 · Node.js · Nginx · Docker Compose · OpenSSL
Repository: github.com/Ayman6594/VPN-Project
Status: **Running on Docker Desktop (Windows)**